TOPDEVCREW · PRUEBA TÉCNICA

Portal de consulta de pólizas y facturas con RAG: ingesta asíncrona, citas verificables y trazabilidad de punta a punta

Ingeniero AI Full Stack Semi-Senior · Entrégala antes del 5 de noviembre de 2026 a las 23:19 UTC

Cómo funciona

- Tienes 72 horas desde que te enviamos este proyecto.
- Puedes usar IA generativa (Claude, Copilot, ChatGPT…) como quieras.
- Solo puedes entregar UNA VEZ. Al pegar el link, congelamos el último commit de tu repositorio en ese momento. Esa es la versión que evaluamos y la que revisamos en la entrevista; los commits posteriores se ignoran y no hay una segunda entrega. Asegúrate de que el repositorio esté listo antes de pegar el link.
- Sube tu trabajo a UN repositorio PÚBLICO de GitHub. Si el proyecto tiene varias partes (backend, frontend, infraestructura), van juntas en ese único repositorio (monorepo). No aceptamos repositorios privados, varios repositorios ni archivos zip.
- En la entrevista técnica vas a recorrer tu código en tu propia pantalla y defender tus decisiones. Entrega solo lo que entiendas.
- Donde algo sea ambiguo, decide y explica por qué en DECISIONS.md.

El proyecto

Seguros Faro del Sur es una aseguradora ficticia cuyos agentes de siniestros pierden horas buscando cláusulas en pólizas en PDF y cotejando facturas de talleres. Te piden un prototipo con tres piezas. Primera: un backend FastAPI que recibe PDF y los procesa en segundo plano con Celery y Redis. El procesamiento extrae texto (Docling es opcional, puedes usar otra librería), divide en fragmentos, genera embeddings, los guarda en PostgreSQL con pgvector y extrae campos estructurados (número de póliza, vigencia, coberturas, monto de factura) validados con Pydantic. Segunda: un endpoint de preguntas en lenguaje natural que responde con RAG y cita documento y página de cada afirmación. Tercera: una interfaz React donde el agente sube documentos, ve el estado del procesamiento, hace preguntas y revisa los campos extraídos. Todo el flujo, desde la petición HTTP hasta la tarea Celery y la llamada al LLM, debe quedar trazado con OpenTelemetry. El acceso al LLM debe ir detrás de una interfaz propia o de LiteLLM para poder cambiar de proveedor (por ejemplo Gemini) y usar un proveedor falso en las pruebas.

REQUISITOS

- POST /documents acepta un PDF, lo guarda, crea un registro con estado (pending, processing, done, failed) y encola una tarea Celery con Redis como broker. Devuelve el id de inmediato.
- La tarea extrae texto por página, genera fragmentos con metadatos (documento, página), calcula embeddings y los persiste en PostgreSQL con pgvector. Reprocesar el mismo documento no debe duplicar fragmentos.
- Extracción estructurada de campos clave con salida validada por modelos Pydantic. Si el LLM devuelve algo inválido, se reintenta de forma acotada y luego se marca el campo o el documento con un error explícito.
- POST /ask recibe una pregunta (opcionalmente filtrada por documento) y devuelve una respuesta basada solo en los fragmentos recuperados, con citas (documento y página). Si no hay evidencia suficiente, responde que no puede contestar en vez de inventar.
- GET /documents y GET /documents/{id} exponen estado, campos extraídos y errores para que la UI haga polling o consulte periódicamente.

- Interfaz React con tres vistas: subida y lista de documentos con su estado, chat de preguntas con citas visibles, y detalle de campos extraídos con indicación de los campos fallidos.
- Capa de proveedor de LLM y embeddings intercambiable por configuración (LiteLLM o interfaz propia), con un proveedor falso determinista para desarrollo y pruebas.
- docker compose levanta API, worker Celery, Redis, PostgreSQL con pgvector, frontend y un colector o visor de trazas OpenTelemetry.

NIVEL DE CALIDAD

- Pruebas automatizadas: unitarias para chunking, validación Pydantic y construcción del prompt; integración de la API y la tarea Celery con PostgreSQL real (testcontainers o compose) y el LLM falso; al menos una prueba del frontend. Ninguna prueba debe llamar a un LLM real.
- Manejo de fallos: reintentos con backoff en la tarea, timeouts en llamadas al LLM, tratamiento de PDF corruptos, escaneados sin texto o demasiado grandes, y estados de error visibles en la API y en la UI.
- Observabilidad con OpenTelemetry: una traza única que cruce FastAPI, Redis/Celery y la llamada al LLM (propagación de contexto al worker), con atributos útiles como id de documento, número de fragmentos, latencia y tokens, y logs correlacionados con el trace id.
- Seguridad: validación de tipo y tamaño de archivo, ningún secreto en el repositorio (.env.example), límites de tasa básicos en /ask y mitigación documentada de prompt injection desde el contenido de los PDF.
- Calidad y reproducibilidad: linter y formateo configurados, un workflow de CI que ejecute las pruebas, y arranque documentado con un solo comando.

DECIDE Y JUSTIFICA

- Estrategia de chunking y recuperación (tamaño, solapamiento, búsqueda solo vectorial o híbrida, umbral de relevancia) y cómo decides que no hay evidencia suficiente para responder.
- Idempotencia y consistencia: cómo evitas duplicados y estados a medias si el worker muere o la tarea se reintenta, y qué garantías de entrega aceptas con Celery y Redis.
- Qué hacer con los campos extraídos de baja confianza: rechazarlos, marcarlos para revisión humana o dejarlos pasar con una advertencia, y cómo se refleja en la UI.
- Cómo y dónde usas la IA generativa durante el desarrollo, y qué verificaste a mano, en especial en las partes de React, Celery y OpenTelemetry que te son menos familiares.

ENTREGABLES

- Un único repositorio público de GitHub (monorepo) con backend, frontend e infraestructura (docker compose), cuyo último commit en el momento de tu entrega es la única versión que se evalúa.
- README con instrucciones para ejecutarlo, arquitectura resumida, qué está hecho, qué no y limitaciones conocidas.
- DECISIONS.md que resuelva las preguntas abiertas, explique los trade-offs y documente cómo usaste la IA generativa.
- Pruebas automatizadas (backend y al menos una de frontend) y un workflow de CI que las ejecute.
- Un script o dataset pequeño de ejemplo (2 o 3 PDF ficticios y preguntas de muestra) para que el entrevistador reproduzca una demo, junto con una captura o ejemplo de traza OpenTelemetry.

Entrega

Pega el link de tu repositorio público de GitHub en:

https://app.topdevcrew.com/test/9cf34b21ff41238e9ba74d3c56fb135c